Fondamentaux de la création d'un module personnalisé Odoo 18

1. Introduction

Odoo est un ERP modulaire basé sur Python, PostgreSQL et XML. La création d'un module personnalisé permet d'adapter Odoo précisément aux besoins métiers d'une entreprise. Ce document présente de manière **complète et structurée** les bases nécessaires pour développer un module sous **Odoo 18**.

---

2. Architecture générale d'Odoo

Odoo repose sur une architecture **MVC étendue** :

- **Models (Python)** : logique métier et structure des données
- **Views (XML)** : interface utilisateur (formulaire, liste, kanban…)
- **Controllers (Python)** : routes HTTP (web, API)
- **Data (XML / CSV)** : données initiales et configurations

Chaque module est indépendant mais peut dépendre d'autres modules.

---

3. Structure standard d'un module Odoo

Un module est un dossier placé dans un chemin déclaré dans `addons_path`.

```
mon_module/
├── __init__.py
├── __manifest__.py
├── models/
│   ├── __init__.py
│   └── mon_model.py
├── views/
│   └── mon_model_views.xml
├── security/
│   ├── ir.model.access.csv
│   └── security.xml
├── data/
│   └── data.xml
└── controllers/
    └── main.py
```

Rôle des dossiers

- **models/** : définition des objets métier

- **views/** : définition des interfaces
- **security/** : gestion des droits
- **data/** : données chargées à l'installation
- **controllers/** : logique HTTP (optionnel)

---

4. Le fichier manifest.py

Le fichier `__manifest__.py` décrit le module.

```
{
    "name": "Mon Module",
    "version": "18.0.1.0.0",
    "summary": "Résumé du module",
    "description": "Description détaillée du module",
    "author": "Votre nom",
    "website": "https://exemple.com",
    "category": "Custom",
    "depends": ["base"],
    "data": [
        "security/ir.model.access.csv",
        "views/mon_model_views.xml",
        "data/data.xml"
    ],
    "installable": True,
    "application": False,
}
```

Champs importants

- `depends` : modules requis
- `data` : fichiers chargés à l'installation
- `application` : visibilité dans le menu Apps

---

5. Initialisation Python (init.py)

Racine du module

```
from . import models
```

Dossier models/

```
from . import mon_model
```

Tous les fichiers Python doivent être importés pour être chargés.

6. Création des modèles (ORM Odoo)

Exemple simple

```
from odoo import models, fields, api

class MonModel(models.Model):
    _name = "mon.module"
    _description = "Mon modèle personnalisé"

    name = fields.Char(string="Nom", required=True)
    active = fields.Boolean(default=True)
    date = fields.Date()
    amount = fields.Float()
```

Types de champs courants

- Char, Text
- Integer, Float, Monetary
- Boolean
- Date, Datetime
- Many2one, One2many, Many2many

7. Relations entre modèles

```
partner_id = fields.Many2one(
    "res.partner",
    string="Client"
)
```

```
line_ids = fields.One2many(
    "mon.module.line",
    "parent_id",
    string="Lignes"
)
```

8. Logique métier et API

Méthodes courantes

```
@api.model
def create(self, vals):
    return super().create(vals)
```

```
def action_confirm(self):
    for record in self:
        record.active = False
```

Décorateurs importants

- `@api.model`
- `@api.depends`
- `@api.onchange`
- `@api.constrains`

---

9. Création des vues XML

Vue formulaire

```
<form>
    <sheet>
        <group>
            <field name="name"/>
            <field name="date"/>
        </group>
    </sheet>
</form>
```

Vue liste

```
<tree>
    <field name="name"/>
    <field name="date"/>
</tree>
```

Vue kanban (extrait)

```
<kanban>
    <templates>
        <t t-name="kanban-box">
```

```
            <div>
                <field name="name"/>
            </div>
        </t>
    </templates>
</kanban>
```

---

10. Actions et menus

```
<record id="action_mon_model" model="ir.actions.act_window">
    <field name="name">Mon Modèle</field>
    <field name="res_model">mon.module</field>
    <field name="view_mode">tree,form</field>
</record>

<menuitem id="menu_mon_module_root" name="Mon Module"/>
<menuitem id="menu_mon_model" parent="menu_mon_module_root"
action="action_mon_model"/>
```

---

11. Sécurité et droits d'accès

ir.model.access.csv (obligatoire)

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
access_mon_model,access.mon.model,model_mon_module,,1,1,1,1
```

Règles de sécurité (optionnel)

```
<record id="rule_mon_model" model="ir.rule">
    <field name="name">Règle Mon Modèle</field>
    <field name="model_id" ref="model_mon_module"/>
    <field name="domain_force">[(1,'=',1)]</field>
</record>
```

---

12. Données initiales et configuration

```
<record id="mon_module_demo" model="mon.module">
    <field name="name">Exemple</field>
</record>
```

Utilisé pour : - valeurs par défaut - données de démonstration - paramètres système

---

13. Contrôleurs (optionnel)

```
from odoo import http

class MonController(http.Controller):

    @http.route('/mon_module/test', auth='user', type='json')
    def test(self):
        return {'status': 'ok'}
```

---

14. Règles d'or du développement Odoo 18 (à respecter absolument)

🧠Règle d'or n°1 – Ordre de chargement XML (CRITIQUE)

🔓 **Un menu ne peut référencer qu'une action déjà chargée**.

🔴 L'ordre **doit toujours être** : 1. **Actions** (`ir.actions.act_window`) 2. **Vues** (`ir.ui.view`) 3. **Menus** (`ir.ui.menu`)

🔩Mauvais ordre (provoque une erreur au chargement) :

```
<menuitem action="action_mon_model"/>
<record id="action_mon_model" model="ir.actions.act_window">...</record>
```

🔗Bon ordre :

```
<record id="action_mon_model" model="ir.actions.act_window">...</record>
<menuitem action="action_mon_model"/>
```

🧠Règle d'or n°2 – Aucun modèle sans droits d'accès

🔓 **Tout modèle DOIT avoir au moins une ligne dans** `ir.model.access.csv`.

Sinon :

```
AccessError: You are not allowed to access this document
```

Même pour les modèles techniques ou temporaires.

🧠Règle d'or n°3 – Toujours charger les fichiers Python

Un fichier Python non importé = **code ignoré**.

Vérifier systématiquement : - `__init__.py` à la racine - `__init__.py` dans chaque sous-dossier

---

🧠Règle d'or n°4 – Ne jamais modifier un module standard

🔩Interdit : - Modifier le code source d'un module Odoo officiel

🔗Toujours utiliser : - héritage de modèles - héritage de vues (XPath) - surcharges de méthodes

---

🧠Règle d'or n°5 – Ne jamais utiliser sudo() sans raison valable

`sudo()` contourne : - les règles de sécurité - les règles d'accès

Utiliser **uniquement si nécessaire**, et localement.

🧠Règle d'or n°6 – Un `_name` = une table SQL

Changer `_name` après installation : 🔩casse la base de données

🎯Toujours réfléchir au nom technique dès le départ.

🧠Règle d'or n°7 – Toujours utiliser @api.depends correctement

```
@api.depends('line_ids.amount')
def _compute_total(self):
    for rec in self:
        rec.total = sum(rec.line_ids.mapped('amount'))
```

🔩Dépendances incorrectes = valeurs jamais recalculées

🧠Règle d'or n°8 – Les données dans data/, pas dans models/

🔩Mauvaise pratique : - créer des données par défaut dans `create()`

🔗Bonne pratique : - utiliser `data.xml`

🧠Règle d'or n°9 – Toujours tester en mode développeur

Utiliser :

```
--dev=all
```

Pour : - voir les erreurs XML - comprendre les héritages de vues - analyser les performances

---

🧠Règle d'or n°10 – Lire les logs avant de chercher ailleurs

90 % des erreurs Odoo sont expliquées dans les logs serveur.

---

15. Cycle complet de développement d'un module Odoo 18 (toutes les étapes)

Cette section décrit **l'ensemble des étapes réelles**, de l'idée à la mise en production, telles qu'elles doivent être suivies en environnement professionnel.

😾Étape 1 – Analyse du besoin métier

Objectif : comprendre **le vrai besoin**, pas la solution technique demandée.

À faire : - Identifier les utilisateurs concernés - Définir les cas d'usage - Lister les règles métier - Déterminer si un module standard couvre déjà le besoin

🔑Règle d'or : *Toujours comprendre le métier avant de coder.*

😾Étape 2 – Choix de la stratégie technique

Décider : - Module 100 % custom ? - Extension d'un module existant ? - Simple héritage de vues / modèles ?

À vérifier : - modules dépendants - impact sur la maintenance

---

😾Étape 3 – Conception fonctionnelle et technique

Livrables recommandés : - Schéma des modèles - Liste des champs - Relations entre modèles - États (workflow) - Droits d'accès par profil

🔑Règle d'or : *Un bon schéma évite 10 refactorings.*

😾Étape 4 – Création du squelette du module

Actions : - Créer l'arborescence du module - Rédiger `__manifest__.py` - Initialiser les `__init__.py` - Déclarer les dépendances

---

😾Étape 5 – Développement des modèles (ORM)

Actions : - Créer les modèles - Définir les champs - Ajouter les contraintes - Implémenter la logique métier

Bonnes pratiques : - noms techniques stables - méthodes courtes - pas de logique métier dans les vues

---

😾Étape 6 – Mise en place de la sécurité

Obligatoire AVANT tout test : - `ir.model.access.csv` - groupes de sécurité - règles record (si besoin)

🔒 Règle d'or : *Un module sans sécurité est un module cassé.*

😾Étape 7 – Création des vues

Ordre recommandé : 1. vues formulaire 2. vues liste 3. vues kanban 4. vues search

Bonnes pratiques : - interfaces simples - groupes logiques - éviter les vues surchargées

---

😾Étape 8 – Actions et menus

🔍Respect strict de l'ordre de chargement :

1. Actions (`ir.actions.act_window`)
2. Vues
3. Menus

Tester immédiatement dans l'interface.

😾Étape 9 – Données initiales et configuration

Utiliser `data.xml` pour : - paramètres par défaut - données techniques - configurations initiales

🔩Ne jamais créer de données dans `create()`.

😾Étape 10 – Tests fonctionnels

À tester : - création / modification / suppression - règles de sécurité - comportements selon les profils - cas limites

Outils : - mode développeur - logs serveur

😾Étape 11 – Optimisation et performance

Vérifications : - boucles ORM inutiles - appels `search()` multiples - champs calculés mal dépendants

🔑Règle d'or : *Ce qui marche peut être lent.*

😾Étape 12 – Documentation

À produire : - documentation utilisateur - documentation technique - commentaires de code utiles

---

😾Étape 13 – Préparation à la livraison

Checklist : - module installable sans erreur - données propres - droits vérifiés - code versionné (Git) - nommage propre

---

😾Étape 14 – Déploiement

Actions : - installation sur environnement de test - validation utilisateur - installation en production

Toujours prévoir un **rollback**.

😾Étape 15 – Maintenance et évolutions

Après livraison : - corrections - évolutions fonctionnelles - compatibilité versions futures

🔦Un module Odoo vit dans le temps.

---

16. Anti-patterns Odoo (ce qu'il ne faut JAMAIS faire)

Cette section liste les **erreurs critiques et mauvaises pratiques** observées en production. Elles provoquent bugs, pertes de données, failles de sécurité ou modules impossibles à maintenir.

👈Anti-pattern n°1 – Modifier le code d'un module standard

🔩Modifier directement un module Odoo officiel

Conséquences : - mises à jour impossibles - conflits majeurs - dette technique élevée

🔗Toujours utiliser : - héritage de modèles - héritage de vues (XPath) - surcharge de méthodes

---

👈Anti-pattern n°2 – Utiliser `sudo()` partout

🔩Exemple dangereux :

```
self.sudo().write({'state': 'done'})
```

Conséquences : - contournement de la sécurité - violations des règles d'accès

🔗Bonne pratique : - utiliser `sudo()` uniquement si nécessaire - le limiter à une opération précise

---

👈Anti-pattern n°3 – Créer des données métier dans `create()`

🔩Mauvaise pratique : - créer des enregistrements par défaut dans `create()`

Conséquences : - doublons - comportements imprévisibles

🔗Utiliser : - `data.xml` - paramètres système

👈Anti-pattern n°4 – Oublier les droits d'accès

🔩Modèle sans `ir.model.access.csv`

Résultat :

```
AccessError: You are not allowed to access this document
```

🔗Toujours définir les droits **avant** de tester.

👈Anti-pattern n°5 – Mettre de la logique métier dans les vues

🔩Exemple : - conditions complexes dans `attrs`

Conséquences : - vues illisibles - logique dupliquée

🔗La logique métier doit rester en Python.

👈Anti-pattern n°6 – Champs calculés sans `@api.depends`

🔩Dépendances manquantes ou incorrectes

Conséquences : - valeurs jamais recalculées - données incohérentes

🔗Toujours déclarer toutes les dépendances.

👈Anti-pattern n°7 – Boucles ORM imbriquées

🔩Exemple :

```
for rec in records:
    lines = self.env['x.line'].search([('parent_id','=',rec.id)])
```

Conséquences : - performances catastrophiques

🔗Utiliser : - relations ORM - `mapped()` - `read_group()`

👈Anti-pattern n°8 – Changer `_name` après installation

🔩Renommer un modèle déjà installé

Conséquences : - base de données cassée - migrations complexes

🔗Le nom technique est **définitif**.

👈Anti-pattern n°9 – Mettre trop de logique dans un seul module

🔩Module "fourre-tout"

Conséquences : - maintenance difficile - dépendances complexes

🔗Découper en modules fonctionnels.

👈Anti-pattern n°10 – Ignorer les logs serveur

🔩Chercher des solutions sans lire les logs

Conséquences : - perte de temps

🔗Les logs expliquent 90 % des erreurs Odoo.

---

17. Tests unitaires Odoo

😸TransactionCase

Utilisé pour tester des fonctionnalités complètes avec rollback automatique en fin de test.

Cas d'usage : - tests fonctionnels - tests multi-modèles

```
from odoo.tests.common import TransactionCase

class TestMonModule(TransactionCase):

    def test_creation(self):
        rec = self.env['mon.module'].create({'name': 'Test'})
        self.assertTrue(rec)
```

😸SavepointCase

Plus rapide, idéal pour de gros volumes de tests.

Cas d'usage : - tests unitaires purs - performances

```
from odoo.tests.common import SavepointCase

class TestMonModuleFast(SavepointCase):

    @classmethod
    def setUpClass(cls):
        super().setUpClass()
        cls.model = cls.env['mon.module']

    def test_name_required(self):
        with self.assertRaises(Exception):
            self.model.create({})
```

---

18. Patterns recommandés Odoo

🖕 Service Layer

Objectif : sortir la logique métier lourde des modèles.

```
class MonService:

    @staticmethod
    def calcul_total(records):
        return sum(records.mapped('amount'))
```

Utilisé pour : - calculs complexes - intégrations externes

🖕 Mixins

Réutilisation de comportements communs.

```
class AuditMixin(models.AbstractModel):
    _name = 'audit.mixin'

    active = fields.Boolean(default=True)
```

🖕 Abstract Models

Base technique non instanciable.

```
class BaseDocument(models.AbstractModel):
    _name = 'base.document'

    name = fields.Char()
```

---

19. Architecture projet multi-modules réelle

⛑️Organisation recommandée

```
addons/
├── business_core/
├── business_sale/
├── business_purchase/
├── business_account/
└── business_reporting/
```

Principes

- un module = une responsabilité
- dépendances claires
- pas de module monolithique

---

20. Conclusion finale

Ce document couvre désormais : - fondamentaux - règles d'or - anti-patterns - cycle complet - tests - patterns avancés - architecture réelle

🔔Niveau **Lead Developer / Architecte Odoo 18**.